0070. Climbing Stairs

Category: 1-D Dynamic Programming | LeetCode: https://leetcode.com/problems

Problem

- You climb 1 or 2 steps at a time. How many distinct ways reach step n?

Intuition

- The last move onto step n came from n-1 or n-2, so ways(n) = ways(n-1) + ways(n-2): Fibonacci.
- Only the previous two values are needed.

Brute Force

- Plain recursion without memo: O(2^n).

Optimal Approach

- A = 1 (ways to reach step 0), b = 1 (step 1).
- Repeat n-1 times: (a, b) = (b, a + b).
- Return b.

Worked Example

```
n = 5.

step: 0 1 2 3 4 5
ways: 1 1 2 3 5 8
```

Java Solution

```java
public class ClimbingStairs {
    public int climbStairs(int n) {
        int a = 1, b = 1;
        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Base cases: n = 1 → 1, n = 2 → 2.